

tf.compat.v1.map_fn Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/compat/v1/map_fn

*Transforms elems by applying fn to each element unstacked on axis 0.
(deprecated arguments)*

Function Signatures

```
tf.compat.v1.map_fn( fn, elems, dtype=None,  
parallel_iterations=None, back_prop=True, swap_memory=False,  
infer_shape=True, name=None, fn_output_signature=None )  
(dtype)
```

Code Examples

```
tf.compat.v1.map_fn(  
fn,  
elems,  
dtype=None,  
parallel_iterations=None,  
back_prop=True,  
swap_memory=False,  
infer_shape=True,  
name=None,  
fn_output_signature=None  
)
```

```
tf.compat.v1.map_fn(  
    fn,  
    elems,  
    dtype=None,  
    parallel_iterations=None,  
    back_prop=True,  
    swap_memory=False,  
    infer_shape=True,  
    name=None,  
    fn_output_signature=None  
)
```

```
tf.map_fn(fn=lambda t: tf.range(t, t + 3), elems=tf.constant([3, 5,  
2]))
```

```
tf.map_fn(fn=lambda t: tf.range(t, t + 3), elems=tf.constant([3, 5,  
2]))
```

```
array([[3, 4, 5],
```

```
[2, 3, 4]], dtype=int32)>
```

Documentation

Transforms `elems` by applying `fn` to each element unstacked on axis 0.
(deprecated arguments)

Used in the notebooks

- Classify Flowers with Transfer Learning

See also `tf.scan`.

`map_fn` unstacks `elems` on axis 0 to obtain a sequence of elements;
calls `fn` to transform each element; and then stacks the transformed
values back together.

Mapping functions with single-Tensor inputs and outputs

If `elems` is a single tensor and `fn`'s signature is `tf.Tensor->tf.Tensor`,
then `map_fn(fn, elems)` is equivalent to
`tf.stack([fn(elem) for elem in tf.unstack(elems)])`. E.g.:

`map_fn(fn, elems).shape = [elems.shape[0]] + fn(elems[0]).shape`.

Mapping functions with multi-arity inputs and outputs

`map_fn` also supports functions with multi-arity inputs and outputs:

- If `elems` is a tuple (or nested structure) of tensors, then those tensors

must all have the same outer-dimension size (`num_elems`); and `fn` is
used to transform each tuple (or structure) of corresponding slices from

elems. E.g., if elems is a tuple (t1, t2, t3), then fn is used to transform each tuple of slices (t1[i], t2[i], t3[i]) (where $0 \leq i < \text{num_elems}$).

- If fn returns a tuple (or nested structure) of tensors, then the

result is formed by stacking corresponding elements from those structures.

If elems is a tuple (or nested structure) of tensors, then those tensors must all have the same outer-dimension size (num_elems); and fn is used to transform each tuple (or structure) of corresponding slices from elems. E.g., if elems is a tuple (t1, t2, t3), then fn is used to transform each tuple of slices (t1[i], t2[i], t3[i]) (where $0 \leq i < \text{num_elems}$).

If fn returns a tuple (or nested structure) of tensors, then the result is formed by stacking corresponding elements from those structures.

Specifying fn's output signature

If fn's input and output signatures are different, then the output signature must be specified using fn_output_signature. (The input and output signatures are different if their structures, dtypes, or tensor types do not match). E.g.:

fn_output_signature can be specified using any of the following:

- A tf.DType or tf.TensorSpec (to describe a tf.Tensor)
- A tf.RaggedTensorSpec (to describe a tf.RaggedTensor)
- A tf.SparseTensorSpec (to describe a tf.sparse.SparseTensor)
- A (possibly nested) tuple, list, or dict containing the above types.

RaggedTensors

`map_fn` supports `tf.RaggedTensor` inputs and outputs. In particular:

- If `elems` is a `RaggedTensor`, then `fn` will be called with each

row of that ragged tensor.

If `elems` has only one ragged dimension, then the values passed to `fn` will be `tf.Tensors`.

If `elems` has multiple ragged dimensions, then the values passed to `fn` will be `tf.RaggedTensors` with one fewer ragged dimension.

- If `elems` has only one ragged dimension, then the values passed to

`fn` will be `tf.Tensors`.

- If `elems` has multiple ragged dimensions, then the values passed to

`fn` will be `tf.RaggedTensors` with one fewer ragged dimension.

- If the result of `map_fn` should be a `RaggedTensor`, then use a

`tf.RaggedTensorSpec` to specify `fn_output_signature`.

If `fn` returns `tf.Tensors` with varying sizes, then use a `tf.RaggedTensorSpec` with `ragged_rank=0` to combine them into a single ragged tensor (which will have `ragged_rank=1`).

If `fn` returns `tf.RaggedTensors`, then use a `tf.RaggedTensorSpec` with the same `ragged_rank`.

- If `fn` returns `tf.Tensors` with varying sizes, then use a

`tf.RaggedTensorSpec` with `ragged_rank=0` to combine them into a single ragged tensor (which will have `ragged_rank=1`).

- If `fn` returns `tf.RaggedTensors`, then use a `tf.RaggedTensorSpec`

with the same `ragged_rank`.

If `elems` is a `RaggedTensor`, then `fn` will be called with each row of that ragged tensor.

- If `elems` has only one ragged dimension, then the values passed to

`fn` will be `tf.Tensors`.

- If `elems` has multiple ragged dimensions, then the values passed to

`fn` will be `tf.RaggedTensors` with one fewer ragged dimension.

If the result of `map_fn` should be a `RaggedTensor`, then use a `tf.RaggedTensorSpec` to specify `fn_output_signature`.

- If `fn` returns `tf.Tensors` with varying sizes, then use a

`tf.RaggedTensorSpec` with `ragged_rank=0` to combine them into a single ragged tensor (which will have `ragged_rank=1`).

- If `fn` returns `tf.RaggedTensors`, then use a `tf.RaggedTensorSpec`

with the same `ragged_rank`.

- `tf.ragged.map_flat_values(fn, rt)`

(if `fn` is expressible as TensorFlow ops)

- `rt.with_flat_values(map_fn(fn, rt.flat_values))`

(otherwise)

SparseTensors

`map_fn` supports `tf.sparse.SparseTensor` inputs and outputs. In particular:

- If `elems` is a `SparseTensor`, then `fn` will be called with each row

of that sparse tensor. In particular, the value passed to `fn` will be a `tf.sparse.SparseTensor` with one fewer dimension than `elems`.

- If the result of `map_fn` should be a `SparseTensor`, then use a

`tf.SparseTensorSpec` to specify `fn_output_signature`. The individual `SparseTensors` returned by `fn` will be stacked into a single `SparseTensor` with one more dimension.

If `elems` is a `SparseTensor`, then `fn` will be called with each row of that sparse tensor. In particular, the value passed to `fn` will be a `tf.sparse.SparseTensor` with one fewer dimension than `elems`.

If the result of `map_fn` should be a `SparseTensor`, then use a `tf.SparseTensorSpec` to specify `fn_output_signature`. The individual `SparseTensors` returned by `fn` will be stacked into a single `SparseTensor` with one more dimension.

- If the function is expressible as TensorFlow ops, use:

`tf.sparse.SparseTensor(st.indices, fn(st.values), st.dense_shape)`

- Otherwise, use:

```
tf.sparse.SparseTensor(st.indices, tf.map_fn(fn, st.values),  
st.dense_shape)
```

If the function is expressible as TensorFlow ops, use:

Otherwise, use:

map_fn vs. vectorized operations

`map_fn` will apply the operations used by `fn` to each element of `elems`, resulting in $O(\text{elems.shape}[0])$ total operations. This is somewhat mitigated by the fact that `map_fn` can process elements in parallel. However, a transform expressed using `map_fn` is s